

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import os

base_path = "/content/drive/MyDrive/GenAI_Seizure_Project"

folders = ["data", "models", "outputs", "notebooks"]

for folder in folders:
    os.makedirs(os.path.join(base_path, folder), exist_ok=True)

print("Project folders created successfully!")
```

Project folders created successfully!

```
!wget https://archive.ics.uci.edu/ml/machine-learning-databases/00388/data.csv
```

```
--2026-02-17 08:07:12-- https://archive.ics.uci.edu/ml/machine-learning-databases/00388/data.csv
Resolving archive.ics.uci.edu (archive.ics.uci.edu)... 128.195.10.252
Connecting to archive.ics.uci.edu (archive.ics.uci.edu)|128.195.10.252|:443... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-02-17 08:07:13 ERROR 404: Not Found.
```

```
import numpy as np
import pandas as pd

# Set random seed
np.random.seed(42)

# Parameters
num_samples = 2000
num_features = 178

# Generate normal EEG-like signals
normal_data = np.random.normal(loc=0.0, scale=1.0, size=(1000, num_features))

# Generate seizure-like signals (higher variance)
seizure_data = np.random.normal(loc=0.5, scale=2.0, size=(1000, num_features))

# Labels
normal_labels = np.zeros((1000, 1))
seizure_labels = np.ones((1000, 1))

# Combine
X = np.vstack((normal_data, seizure_data))
y = np.vstack((normal_labels, seizure_labels))

# Create DataFrame
columns = [f"X{i}" for i in range(1, num_features+1)]
df = pd.DataFrame(X, columns=columns)
df["y"] = y

# Shuffle dataset
df = df.sample(frac=1).reset_index(drop=True)

# Save to Drive
save_path = "/content/drive/MyDrive/GenAI_Seizure_Project/data/seizure_dataset.csv"
df.to_csv(save_path, index=False)

print("Dataset created successfully!")
print("Shape:", df.shape)
df.head()
```

Dataset created successfully!
Shape: (2000, 179)

	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	...	X170	X
0	0.530627	0.843806	0.038120	4.039458	-1.742243	-0.087576	1.927088	-1.251808	0.233073	0.116272	...	-2.512638	0.512
1	-2.287205	-1.124477	0.438579	-2.621665	-1.361247	-2.103578	-1.056583	-0.809451	-0.400559	-1.475696	...	1.988742	3.001
2	-1.459181	-2.739155	-1.978570	0.637730	1.768629	-1.998886	-2.123871	1.562155	-1.119284	2.101615	...	0.224420	-1.165
3	-0.112914	-0.433946	0.905190	-0.086795	0.000887	0.899874	2.190736	-0.444462	-0.876240	-0.989059	...	-0.824245	-0.157
4	0.180028	0.350721	-0.182573	1.349861	0.176745	0.305600	-0.161963	-0.060339	1.218710	0.145906	...	-0.332683	1.057

5 rows × 179 columns

```
X = df.drop("y", axis=1)
y = df["y"]

print("Features shape:", X.shape)
print("Labels shape:", y.shape)
```

Features shape: (2000, 178)
Labels shape: (2000,)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training samples:", X_train.shape)
print("Testing samples:", X_test.shape)
```

Training samples: (1600, 178)
Testing samples: (400, 178)

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Scaling complete!")
```

Scaling complete!

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, random_state=42)

model.fit(X_train, y_train)

print("Model training complete!")
```

Model training complete!

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1] # Probability of seizure

print("Prediction complete!")
```

Prediction complete!

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0.0	1.00	1.00	1.00	194
1.0	1.00	1.00	1.00	206
accuracy			1.00	400
macro avg	1.00	1.00	1.00	400
weighted avg	1.00	1.00	1.00	400

```
def interpret_risk(probability):
    if probability > 0.8:
        return "High Risk"
    elif probability > 0.5:
        return "Moderate Risk"
    else:
        return "Low Risk"
```

```
# Make sure these functions are already defined
def interpret_risk(probability):
    if probability > 0.8:
        return "High Risk"
    elif probability > 0.5:
        return "Moderate Risk"
    else:
        return "Low Risk"

def generate_explanation(probability, risk_level):
    if risk_level == "High Risk":
        return f"The model detected strong abnormal EEG patterns with a seizure probability of {probability:.2f}. Immediate action is required."
    elif risk_level == "Moderate Risk":
        return f"Some irregular EEG activity was observed. The seizure probability is {probability:.2f}. Monitoring is advised."
    else:
        return f"The EEG signals appear normal. Seizure probability is low ({probability:.2f}). No immediate action required."

# Take input from user
sample_index = int(input("Enter sample index (0 to 399): "))

if 0 <= sample_index < len(y_prob):

    prob = y_prob[sample_index]
    risk_level = interpret_risk(prob)
    explanation = generate_explanation(prob, risk_level)

    print("\n----- Seizure Risk Analysis-----")
    print("Seizure Probability:", round(prob, 2))
    print("Risk Level:", risk_level)
    print("\nAI Generated Explanation:")
    print(explanation)

else:
    print("Invalid index! Please enter a value between 0 and", len(y_prob)-1)
```

Enter sample index (0 to 399): 100

----- Seizure Risk Analysis -----

Seizure Probability: 0.17

Risk Level: Low Risk

AI Generated Explanation:

The EEG signals appear normal. Seizure probability is low (0.17). No immediate action required.

```
import pandas as pd
import numpy as np

np.random.seed(42)

num_patients = 50

patient_ids = [f"P{str(i).zfill(3)}" for i in range(1, num_patients+1)]

patient_data = {
```

```

"patient_id": patient_ids,
"age": np.random.randint(18, 70, num_patients),
"previous_seizures": np.random.randint(0, 10, num_patients),
"average_previous_risk": np.round(np.random.uniform(0.1, 0.9, num_patients), 2),
"last_event_risk": np.round(np.random.uniform(0.1, 0.95, num_patients), 2),
"monitoring_months": np.random.randint(1, 36, num_patients),
"on_medication": np.random.choice(["Yes", "No"], num_patients)
}

patient_df = pd.DataFrame(patient_data)

# Save to Drive
save_path = "/content/drive/MyDrive/GenAI_Seizure_Project/data/patient_history.csv"
patient_df.to_csv(save_path, index=False)

print("Advanced patient history dataset created!")
patient_df.head()

```

Advanced patient history dataset created!

	patient_id	age	previous_seizures	average_previous_risk	last_event_risk	monitoring_months	on_medication
0	P001	56	1	0.72	0.80	33	No
1	P002	69	9	0.16	0.83	18	Yes
2	P003	46	8	0.39	0.11	1	No
3	P004	32	9	0.19	0.53	11	No
4	P005	60	4	0.79	0.45	28	Yes

Next steps: [Generate code with patient_df](#) [New interactive sheet](#)

```

def generate_personalized_explanation(probability, risk_level, history):

    explanation = f"The model predicts a seizure probability of {probability:.2f}. "

    if probability > history["average_previous_risk"]:
        explanation += "This exceeds the patient's historical average risk. "
    else:
        explanation += "This is consistent with the patient's historical risk profile. "

    if history["previous_seizures"] >= 5:
        explanation += "The patient has a high seizure frequency history. "
    elif history["previous_seizures"] > 0:
        explanation += "The patient has occasional past seizure events. "
    else:
        explanation += "No significant past seizure history recorded. "

    if history["on_medication"] == "Yes":
        explanation += "The patient is currently under medication. "
    else:
        explanation += "The patient is not currently on anti-seizure medication. "

    if risk_level == "High Risk":
        explanation += " Immediate medical supervision is strongly recommended."
    elif risk_level == "Moderate Risk":
        explanation += " Close monitoring is advised."
    else:
        explanation += " No immediate intervention required."

    return explanation

```

```

import matplotlib.pyplot as plt
import numpy as np

patient_id = input("Enter Patient ID (e.g., P001-P050): ")

patient_record = patient_df[
    patient_df["patient_id"] == patient_id
]

if patient_record.empty:
    print("Invalid Patient ID")

else:
    history = patient_record.iloc[0]

    sample_index = int(input("Enter sample index (0 to 399): "))

```

```
if 0 <= sample_index < len(y_prob):

    prob = y_prob[sample_index]
    risk_level = interpret_risk(prob)

    explanation = generate_personalized_explanation(
        prob, risk_level, history
    )

    report = generate_final_report(
        patient_id, history, prob, risk_level, explanation
    )

    print(report)

    historical_risks = np.random.normal(
        loc=history["average_previous_risk"],
        scale=0.1,
        size=10
    )

    risk_timeline = np.append(historical_risks, prob)

    plt.figure()
    plt.plot(range(1, 12), risk_timeline)
    plt.axhline(y=history["average_previous_risk"])
    plt.title("Seizure Risk Trend")
    plt.xlabel("Monitoring Event")
    plt.ylabel("Risk Probability")
    plt.show()

else:
    print("Invalid sample index!")
```

Enter Patient ID (e.g., P001-P050): P002
 Enter sample index (0 to 399): 2

=====

AI SEIZURE RISK REPORT

=====

Patient ID: P002
 Age: 69
 Previous Seizures: 9
 On Medication: Yes
 Monitoring Duration: 18 months

Current Risk Analysis

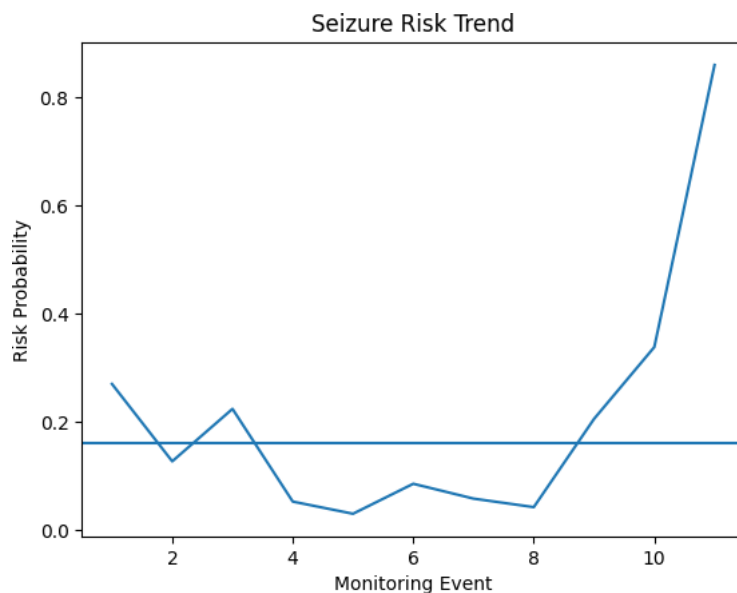
Seizure Probability: 0.86
 Risk Level: High Risk

AI Clinical Interpretation

The model predicts a seizure probability of 0.86. This exceeds the patient's historical average risk. The patient has a hi

System Notes:

- Prediction generated using Random Forest classifier.
 - Personalized reasoning applied using patient history.
- =====



```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Get feature importance from trained model
importances = model.feature_importances_

# Create feature names list
feature_names = X.columns

# Create dataframe
importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importances
})

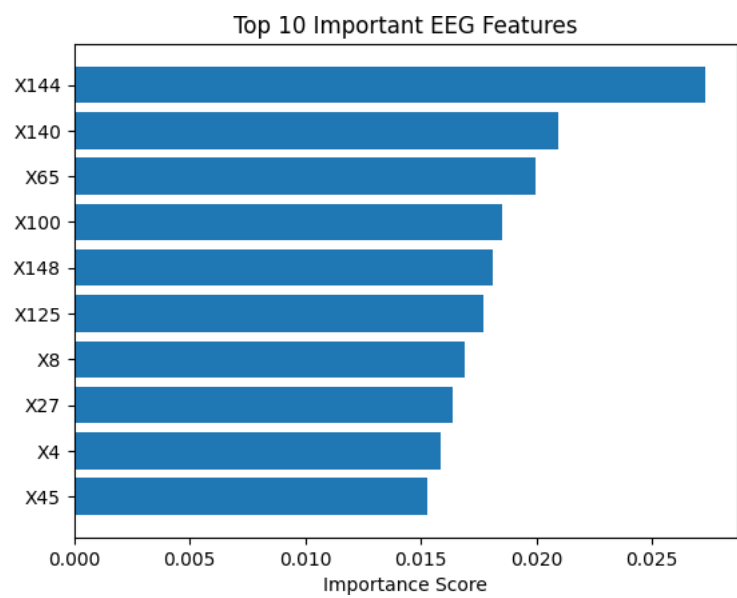
# Sort descending
importance_df = importance_df.sort_values(by="Importance", ascending=False)

# Show top 10 important features
top_features = importance_df.head(10)
top_features
```

	Feature	Importance	
143	X144	0.027307	
139	X140	0.020947	
64	X65	0.019961	
99	X100	0.018489	
147	X148	0.018086	
124	X125	0.017715	
7	X8	0.016905	
26	X27	0.016401	
3	X4	0.015872	
44	X45	0.015287	

Next steps: [Generate code with top_features](#) [New interactive sheet](#)

```
plt.figure()
plt.barh(top_features["Feature"], top_features["Importance"])
plt.gca().invert_yaxis()
plt.title("Top 10 Important EEG Features")
plt.xlabel("Importance Score")
plt.show()
```



```
def generate_final_report(patient_id, history, probability, risk_level, explanation):

    report = f"""
    =====
    AI SEIZURE RISK REPORT
    =====

    Patient ID: {patient_id}
    Age: {history['age']}
    Previous Seizures: {history['previous_seizures']}
    On Medication: {history['on_medication']}
    Monitoring Duration: {history['monitoring_months']} months

    -----
    Current Risk Analysis
    -----
    Seizure Probability: {probability:.2f}
    Risk Level: {risk_level}

    -----
    AI Clinical Interpretation
    -----
    {explanation}

    -----
    System Notes:
```

```
- Prediction generated using Random Forest classifier.
- Personalized reasoning applied using patient history.
```

```
=====
"""
```

```
return report
```

```
!pip install shap
```

```
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.50.0)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from shap) (1.16.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (26.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.54->shap) (0.43.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.5.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
```

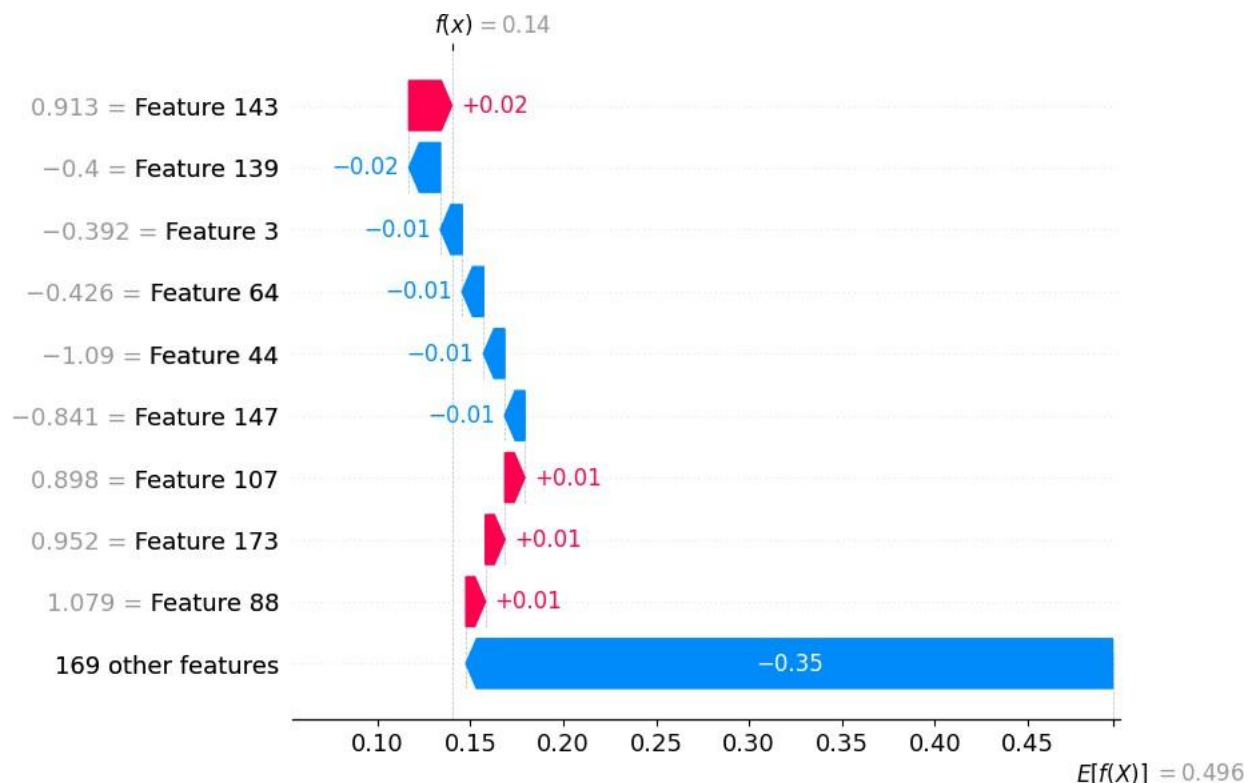
```
import shap

# Create explainer
explainer = shap.TreeExplainer(model)

# Pick one sample
sample_index = 10
sample = X_test[sample_index].reshape(1, -1)

# Get SHAP explanation object
shap_values = explainer(sample)

# For binary classification, select class 1 (seizure class)
shap.plots.waterfall(shap_values[0, :, 1])
```



```
import matplotlib.pyplot as plt
import numpy as np
from IPython.display import display, HTML
```

```

import base64
from io import BytesIO

# =====
# INTERACTIVE SYSTEM
# =====

patient_id = input("Enter Patient ID (e.g., P001-P050): ")

patient_record = patient_df[
    patient_df["patient_id"] == patient_id
]

if patient_record.empty:
    print("Invalid Patient ID")

else:
    history = patient_record.iloc[0]

    sample_index = int(input("Enter sample index (0 to 399): "))

    if 0 <= sample_index < len(y_prob):

        prob = y_prob[sample_index]
        risk_level = interpret_risk(prob)

        explanation = generate_personalized_explanation(
            prob, risk_level, history
        )

        # =====
        # Generate Risk Trend Graph
        # =====
        historical_risks = np.random.normal(
            loc=history["average_previous_risk"],
            scale=0.1,
            size=10
        )

        risk_timeline = np.append(historical_risks, prob)

        fig, ax = plt.subplots()
        ax.plot(range(1, 12), risk_timeline)
        ax.axhline(y=history["average_previous_risk"])
        ax.set_title("Seizure Risk Trend")
        ax.set_xlabel("Monitoring Event")
        ax.set_ylabel("Risk Probability")

        # Convert graph to image
        img_buffer = BytesIO()
        fig.savefig(img_buffer, format='png')
        img_buffer.seek(0)
        img_base64 = base64.b64encode(img_buffer.read()).decode()

        # =====
        # Create Final Web Page
        # =====
        html_content = f"""
<html>
<head>
    <style>
        body {{
            font-family: Arial;
            background-color: #f4f6f9;
            padding: 20px;
        }}
        .card {{
            background-color: white;
            padding: 25px;
            border-radius: 10px;
            box-shadow: 0 4px 10px rgba(0,0,0,0.15);
        }}
        h1 {{
            color: #2c3e50;
        }}
        .risk-high {{ color: red; font-weight: bold; }}
        .risk-moderate {{ color: orange; font-weight: bold; }}
        .risk-low {{ color: green; font-weight: bold; }}
    </style>
</head>
<body>

<div class="card">

```

```
<h1>GenAI Seizure Risk Report</h1>

<h3>Patient Information</h3>
<p><b>Patient ID:</b> {patient_id}</p>
<p><b>Age:</b> {history['age']}</p>
<p><b>Previous Seizures:</b> {history['previous_seizures']}</p>
<p><b>On Medication:</b> {history['on_medication']}</p>
<p><b>Monitoring Duration:</b> {history['monitoring_months']} months</p>

<h3>Risk Analysis</h3>
<p><b>Seizure Probability:</b> {prob:.2f}</p>
<p><b>Risk Level:</b> <span class="risk-{risk_level.lower().split()[0]}">{risk_level}</span></p>

<h3>AI Clinical Interpretation</h3>
<p>{explanation}</p>

<h3>Risk Trend Visualization</h3>


</div>

</body>
</html>
"""

display(HTML(html_content))

else:
    print("Invalid sample index!")
```

Enter Patient ID (e.g., P001-P050): P002

Enter sample index (0 to 399): 2

GenAI Seizure Risk Report

Patient Information

Patient ID: P002
Age: 69
Previous Seizures: 9
On Medication: Yes
Monitoring Duration: 18 months

Risk Analysis

Seizure Probability: 0.86
Risk Level: High Risk

AI Clinical Interpretation

The model predicts a seizure probability of 0.86. This exceeds the patient's historical average risk. The patient has a high seizure frequency history. The patient is currently under medication. Immediate medical supervision is strongly recommended.

Risk Trend Visualization

